

Licence d'excellence en intelligence artificielle

Projet de fin d'étude

Plateforme intelligente de gestion et d'analyse automatisée des CV

Réalisé par :

BAHRI ILHAME

BOUNADAR DOUAA

Encadre par :

Mr. Zakary Omar

Année universitaire :

2024 / 2025

Table des matières

Chapitre 1 : Introduction.....	3
1. Contexte général	3
2. Problématique	4
3. Objectifs du projet.....	5
4. Méthodologie adoptée.....	6
5. Gestion du projet.....	8
Chapitre 2 : État de l’art.....	13
1. Introduction.....	13
2. Traitement automatique des CV : Défis et enjeux	13
3. Les systèmes ATS (Applicant Tracking Systems) traditionnels	13
4. Approches basées sur le Machine Learning.....	15
5. Modèles de type NER (Named Entity Recognition).....	17
6. Apport des LLMs (Large Language Models)	19
7. Comparaison des approches.....	20
8. Conclusion	21
Chapitre 3 : Analyse des besoins (Cahier des charges).....	22
1. Introduction.....	22
2. Présentation du système à concevoir	22
3. Acteurs du système	22
4. Besoins fonctionnels (BF).....	23
5. Besoins non fonctionnels (BNF).....	24
6. Contraintes techniques	24
7. Diagramme de cas d’utilisation (Use Case).....	25
8. Conclusion	28

Chapitre 1 : Introduction

1. Contexte général

Le processus de recrutement constitue aujourd'hui un levier stratégique fondamental pour assurer la croissance durable et la compétitivité des entreprises, en particulier dans un marché du travail de plus en plus dynamique et concurrentiel. Attirer, identifier et recruter les meilleurs talents est un défi permanent pour les départements des ressources humaines, qui doivent composer avec des contraintes de temps, de budget et de qualité. Cependant, ce processus, lorsqu'il est réalisé de manière traditionnelle, reste majoritairement manuel, fastidieux et peu optimal. En effet, le tri et l'analyse des candidatures nécessitent un investissement humain considérable, et comportent un risque élevé de biais cognitifs ou de décisions subjectives, pouvant compromettre la sélection des profils les plus adaptés.

Avec la généralisation des plateformes de recrutement en ligne, les entreprises publient désormais leurs offres d'emploi sur divers canaux numériques, ce qui augmente considérablement leur visibilité. Cette large diffusion entraîne un afflux massif de candidatures, parfois plusieurs centaines de CV pour un seul poste. Face à cette volumétrie croissante, les recruteurs peinent à traiter rapidement et objectivement tous les dossiers reçus, ce qui ralentit le processus de sélection et peut entraîner la perte de talents potentiels.

Parallèlement, les avancées spectaculaires dans le domaine de l'intelligence artificielle, et plus particulièrement dans les modèles de langage de grande taille (Large Language Models – LLMs), révolutionnent la manière dont les documents textuels peuvent être analysés et compris. Des modèles tels que GPT (OpenAI), Gemini (Google), Claude (Anthropic) ou encore Mistral (France), sont désormais capables d'interpréter finement le langage naturel, d'extraire des informations pertinentes, de synthétiser des contenus longs et non structurés, et même de détecter des relations sémantiques complexes entre différentes entités textuelles. Cette capacité ouvre la voie à de nouvelles approches d'analyse automatisée, notamment dans le domaine du recrutement.

Dans ce contexte, l'intégration de solutions basées sur l'IA dans les plateformes de gestion des candidatures représente une opportunité majeure pour les entreprises. Une telle solution permettrait d'automatiser le traitement des CV, d'accélérer le tri des profils, de générer des recommandations personnalisées en fonction des besoins du poste, et de réduire les biais dans les décisions de recrutement. En s'appuyant sur la puissance des LLMs, il devient possible de transformer un processus souvent laborieux en une démarche agile, intelligente et plus équitable. Cette modernisation du recrutement contribue non seulement à l'efficacité opérationnelle, mais renforce également l'image d'innovation de l'entreprise auprès des talents qu'elle souhaite attirer.

2. Problématique

Dans le contexte actuel de digitalisation des processus RH, de nombreuses entreprises s'équipent de **systèmes de gestion des candidatures**, appelés **ATS (Applicant Tracking Systems)**. Toutefois, malgré leur adoption croissante, la majorité de ces solutions se limitent à la simple **collecte, centralisation et stockage de fichiers CV au format PDF**, sans analyse sémantique approfondie ni extraction structurée de l'information. Cette approche rudimentaire présente de nombreuses **limitations opérationnelles et stratégiques** :

- **Lecture manuelle chronophage** : Les recruteurs doivent ouvrir et lire chaque CV individuellement, ce qui devient rapidement impraticable lorsque des centaines de candidatures sont reçues pour un même poste. Ce processus augmente le temps de traitement et retarde la prise de décision.
- **Absence d'exploitation intelligente des données** : Les informations clés (expérience, formation, compétences, langues, certifications...) contenues dans les CV ne sont ni extraites ni normalisées. Elles restent inaccessibles à toute forme d'analyse ou de recherche avancée, rendant difficile la création de bases de données exploitables pour des tableaux de bord, du reporting ou de la recherche multicritère.
- **Matching basé sur des mots-clés exacts** : Les systèmes actuels reposent souvent sur une logique de filtrage par mots-clés rigide, sans compréhension du **contexte sémantique**, des **compétences transférables**, ou encore de la **polyvalence d'un profil**. Cela peut conduire à écarter des candidats pertinents simplement parce qu'ils n'ont pas utilisé la terminologie exacte attendue.

Pour répondre à ces limites, une piste consiste à entraîner localement des modèles d'IA spécialisés, par exemple :

- Des modèles de **reconnaissance d'entités nommées (NER)** pour identifier automatiquement les noms, dates, diplômes, compétences...
- Des **classificateurs** pour catégoriser les CV par métier ou niveau d'expérience.
- Des modèles de **matching sémantique** entre offres et candidatures.

Cependant, cette approche soulève plusieurs défis :

- Elle requiert **des datasets annotés** de manière fine, souvent inexistantes ou difficiles à constituer.
- Elle mobilise des **compétences techniques pointues** en data science, NLP (traitement automatique du langage naturel), MLOps...
- Elle nécessite **une infrastructure de calcul puissante** et coûteuse, notamment pour l'entraînement de modèles performants.

Dans ce contexte, une alternative pertinente serait d'exploiter la **puissance des modèles de langage pré-entraînés (LLMs)** accessibles via API, tels que **Gemini (Google)**, **GPT**

(OpenAI), ou Claude (Anthropic). Ces modèles, déjà entraînés sur d'immenses corpus multilingues, sont capables de :

- Lire et comprendre un CV sous forme de texte brut.
- Extraire automatiquement les informations clés de manière contextualisée.
- Évaluer la pertinence d'un profil par rapport à une offre, sur la base de raisonnements sémantiques, et non uniquement de mots-clés.

D'où la question centrale :

Comment tirer parti des capacités avancées des modèles de langage (LLMs), tels que Gemini de Google, pour automatiser l'extraction sémantique des informations contenues dans les CV et améliorer la qualité du matching avec les offres d'emploi, tout en s'affranchissant de l'entraînement local de modèles IA complexes ?

3. Objectifs du projet

Ce projet a pour ambition de concevoir et de développer une **plateforme web intelligente de gestion et d'analyse de CV**, exploitant la puissance des modèles de langage avancés (LLMs) via une API externe, en l'occurrence **Gemini de Google**. L'objectif principal est de **réduire considérablement la charge manuelle liée au traitement des candidatures**, tout en **améliorant la qualité du processus de sélection** grâce à une analyse sémantique fine et automatisée.

Le projet se décline en deux **objectifs fonctionnels majeurs**, complémentaires et interdépendants :

a. Extraction automatique d'informations

Lorsqu'un candidat téléverse son CV au format PDF sur la plateforme, le système effectue automatiquement les opérations suivantes :

- **Extraction du texte brut** : grâce à des scripts Python, le contenu du CV est transformé en texte brut, nettoyé et préparé pour le traitement linguistique.
- **Génération de prompt structuré** : un prompt spécifique est formulé pour interroger l'API Gemini afin d'enrichir et structurer les informations extraites.
- **Analyse sémantique via Gemini** : le texte est envoyé à l'API qui, grâce à ses capacités de compréhension contextuelle, renvoie une réponse formatée contenant les **données clés** suivantes :
 - Informations personnelles : nom, prénom, adresse e-mail, numéro de téléphone ;
 - Parcours académique : diplômes obtenus, établissements, années ;
 - Expérience professionnelle : postes occupés, entreprises, dates de début et de fin, missions réalisées ;

- Compétences techniques : langages de programmation, outils logiciels, frameworks ;
- Compétences linguistiques : langues maîtrisées et niveau.

Cette étape vise à **structurer le contenu non structuré** des CV pour faciliter l'exploitation ultérieure par la plateforme.

b. Matching intelligent

Lorsqu'un recruteur publie une nouvelle offre d'emploi sur la plateforme, le système procède automatiquement aux étapes suivantes :

- **Formulation d'un prompt de comparaison** : un prompt est généré pour demander à Gemini d'analyser le degré de compatibilité sémantique entre le contenu de l'offre et chaque CV préalablement traité.
- **Scoring et classement des profils** : pour chaque candidat, une **note de compatibilité** est calculée (par exemple sur 100), accompagnée d'un **résumé sémantique** justifiant l'évaluation (points forts, éventuelles lacunes, suggestions).
- **Affichage des résultats** : les recruteurs visualisent une **liste triée** des candidats pertinents avec, pour chacun, une fiche d'analyse générée automatiquement.

Ce module vise à **accélérer la présélection, objectiver le processus de recrutement et valoriser les compétences réelles**, y compris les compétences transférables souvent ignorées par les filtres classiques.

4. Méthodologie adoptée

La mise en œuvre du projet repose sur une approche **modulaire, agile et itérative**, centrée sur l'intégration harmonieuse de composants technologiques modernes avec des services d'intelligence artificielle. La plateforme est développée selon une architecture **MERN (MongoDB, Express, React, Node.js)**, enrichie par un module IA externe développé en Python.

Architecture technique adoptée

Composant	Technologie	Rôle
Frontend	React.js + Tailwind CSS	Développement d'une interface utilisateur responsive, intuitive et fluide, adaptée aux deux profils (candidat/recruteur).
Backend	Node.js + Express	Implémentation d'une API REST pour la gestion des utilisateurs, des sessions, des fichiers et des interactions avec l'IA.
Base de données	MongoDB	Stockage des utilisateurs, des offres, des CV bruts et extraits, des scores de matching et des logs IA.
Module IA	Python + Gemini API	Traitement des CV (texte brut), génération et envoi des prompts, parsing des réponses structurées, gestion des logs d'analyse.
Authentification	JWT (JSON Web Token)	Sécurisation des sessions, séparation des rôles (candidat vs recruteur), accès restreint aux fonctionnalités critiques.

Étapes méthodologiques

La démarche de développement s'articule autour des axes suivants :

- **Modularisation du projet** : séparation claire des responsabilités entre le frontend, le backend, et le module IA, facilitant les tests indépendants, la maintenance et l'évolution future.
- **Prompt engineering avancé** : rédaction de prompts précis, dynamiques et contextualisés pour garantir des réponses pertinentes de la part du modèle Gemini, avec des formats de retour exploitables (JSON, tableaux, etc.).
- **Validation manuelle et itérative** : dans un premier temps, les résultats produits par l'IA seront vérifiés manuellement pour identifier les cas d'erreur, ajuster les prompts, et améliorer la fiabilité globale du système.
- **Phase de test sur des données réelles** : plusieurs CV authentiques (de profils variés) et des offres d'emploi issues du marché marocain et international seront utilisés pour évaluer la **performance du système**, en termes :
 - **Qualité d'extraction** (complétude, justesse des données extraites) ;
 - **Pertinence du matching**;
 - **Réduction du temps de traitement** pour les utilisateurs RH.

5. Gestion du projet

La réussite de ce projet repose en grande partie sur une organisation rigoureuse et une collaboration efficace entre les deux membres du binôme. Pour cela, nous avons adopté une approche structurée de gestion de projet inspirée des méthodologies **Agile Scrum**, en mettant l'accent sur l'adaptabilité, la communication continue et l'amélioration progressive. Nous avons également utilisé des **outils collaboratifs** pour assurer le suivi du travail, la répartition des responsabilités, et la coordination quotidienne.

a. Répartition des tâches et collaboration

Nous avons fait le choix stratégique de **ne pas spécialiser les rôles** entre frontend, backend ou traitement IA. Cette approche transversale nous a permis de:

- **Monter en compétences** sur toutes les technologies impliquées dans le projet (React, Node.js, Express, MongoDB, etc.).
- Favoriser un **travail d'équipe équilibré**, où chacun pouvait contribuer aux différents modules.

La répartition des tâches s'est organisée autour de **modules fonctionnels** du projet, par exemple :

- Le module d'authentification et de gestion des utilisateurs.
- Le système de traitement et d'extraction automatique d'informations à partir des CV PDF.
- Le moteur de matching entre profils et offres.
- L'interface RH pour la consultation et le filtrage des profils.

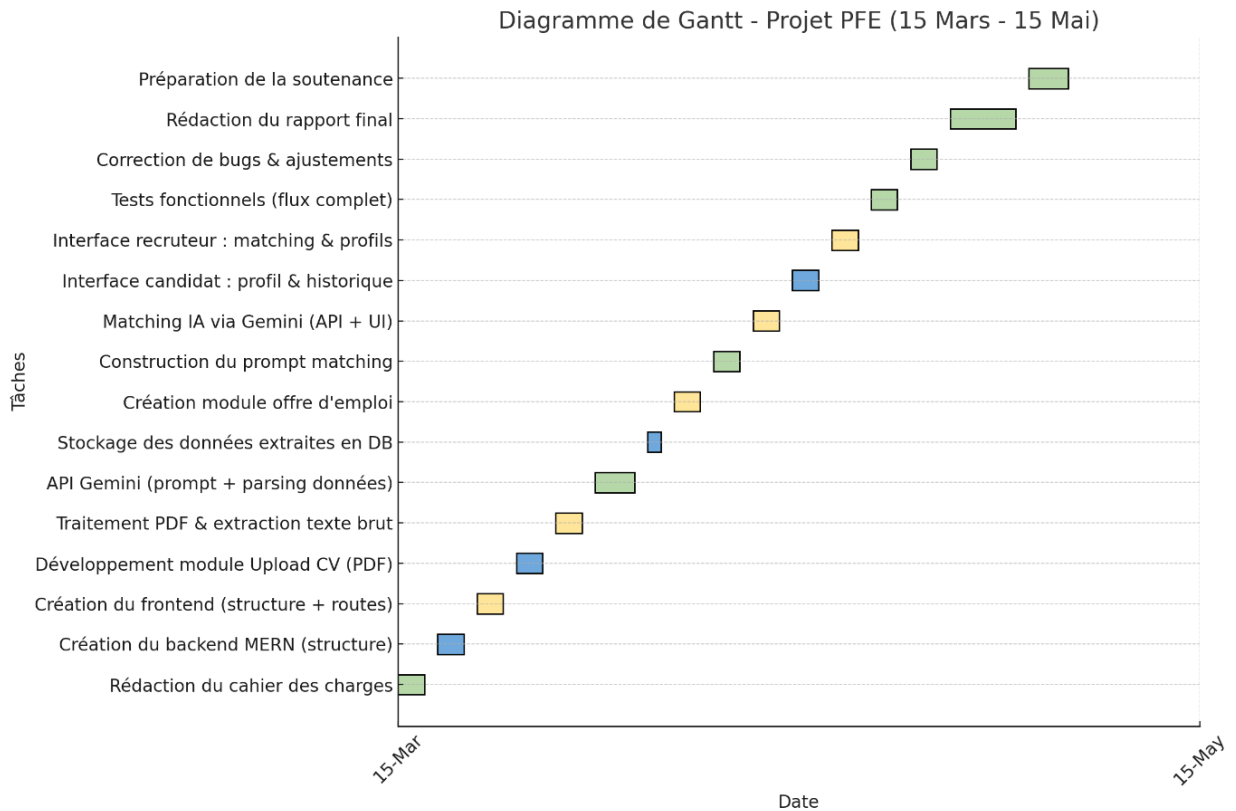
Nous avons travaillé ensemble sur les **étapes critiques**, notamment :

- La **définition de l'architecture globale** de l'application.
- L'**intégration des composants IA**.
- Les tests de l'application de bout en bout.

Chaque début de semaine, nous réalisons une **planification** (type *Sprint Planning*) avec notre encadrant pour :

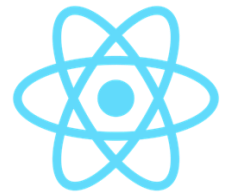
- Identifier les priorités.
- Découper les tâches en *user stories* ou *tickets*.
- Estimer la charge de travail.

Nous avons également mis en place des **réunions journalières rapides** (*daily stand-ups*) pour faire le point sur l'avancement, ajuster le plan si nécessaire, et lever les éventuels blocages.



b. Technologies utilisées

ReactJS est une bibliothèque JavaScript qui permet de créer des interfaces utilisateur dynamiques à l'aide de composants réutilisables. Elle est rapide et efficace grâce à son DOM virtuel, et idéale pour les applications web modernes.



Tailwind CSS est un framework CSS utilitaire qui permet de créer des designs personnalisés rapidement en utilisant des classes pré-définies. Contrairement aux frameworks traditionnels,

Node.js est un environnement d'exécution JavaScript côté serveur, basé sur le moteur V8 de Google Chrome. Il permet d'exécuter du JavaScript en dehors du navigateur, offrant ainsi la possibilité de développer des applications serveur rapides et scalables.



EXPRESS Js

Express.js est un framework minimaliste pour Node.js, conçu pour simplifier le développement d'applications web et d'API. Il offre des outils puissants pour gérer les routes, les requêtes HTTP, les middlewares et les sessions.

Axios est une bibliothèque JavaScript utilisée pour faire des requêtes HTTP depuis le navigateur ou Node.js. Elle simplifie la gestion des requêtes asynchrones, supporte les promesses, et permet de gérer facilement les réponses et erreurs.



MongoDB est une base de données NoSQL orientée document, qui stocke les données au format BSON (une extension de JSON). Contrairement aux bases de données relationnelles, MongoDB permet une plus grande flexibilité dans la gestion des données, ce qui est idéal pour les applications qui nécessitent une évolution rapide du modèle de données.

JWT (JSON Web Token) est un standard ouvert pour sécuriser les échanges de données entre un client et un serveur. Il encode des informations sous forme de token signé, souvent utilisé pour l'authentification et l'autorisation.



FastAPI est un framework web moderne pour Python, conçu pour la création d'API rapides et performantes. Il repose sur une documentation interactive et une grande rapidité d'exécution. FastAPI est particulièrement apprécié pour sa simplicité, sa rapidité de développement et ses performances élevées.

Python est un langage de programmation interprété, de haut niveau et généraliste, caractérisé par sa syntaxe simple et lisible. Il combine typage dynamique, gestion automatique de la mémoire et une vaste bibliothèque standard. Multi-paradigme (orienté objet, impératif, fonctionnel), il privilégie la productivité et la portabilité, tout en maintenant un équilibre entre simplicité et puissance.



a. Outils utilisés

Afin d'assurer une coordination fluide, un suivi rigoureux de l'avancement, et une centralisation des ressources, nous avons mis en place un ensemble d'**outils collaboratifs** adaptés à chaque aspect du projet. Ces outils ont été choisis en fonction de leur facilité d'utilisation, de leur compatibilité avec notre stack technique et de leur capacité à favoriser le travail en binôme, même à distance.

➤ Git et GitHub – Gestion de versions et collaboration

Nous avons utilisé **Git** comme système de contrôle de version, associé à **GitHub** pour l'hébergement du code et la gestion collaborative. Voici comment nous avons structuré notre travail avec ces outils :

- Mise en place d'une **branche principale (main)** contenant tous les versions du projet.

Cette organisation a permis :

- D'**éviter les conflits** de code grâce au travail sur des branches indépendantes.
- D'**améliorer la qualité du code** via la revue mutuelle.
- De **tracer toutes les évolutions**, ce qui facilite la compréhension de l'historique du projet.

➤ Google Docs et Notion – Documentation et planification

Pour tout ce qui concerne la **documentation du projet**, les comptes rendus de réunions, les plans techniques et les idées de conception, nous avons utilisé :

- **Google Docs**, pour sa simplicité, son mode collaboratif en temps réel, et la possibilité de commenter, corriger et partager rapidement.
- **Notion**, comme espace centralisé pour :
 - La **roadmap** du projet (objectifs par semaine),
 - Le découpage des tâches,
 - La documentation technique (description des APIs, architecture, structure des données),
 - Un suivi de nos apprentissages et des ressources utiles.

Cela nous a permis de :

- Garder une **traçabilité écrite** de toutes les décisions prises,
- **Faciliter le passage à l'oral** pour la présentation du projet,
- Maintenir une **vue globale** du projet à chaque étape.



Google Docs



➤ Postman – Tests des API REST

Pour tester et valider le bon fonctionnement de notre **backend** (Node.js + Express), nous avons utilisé **Postman**, un outil indispensable pour :

- Tester les **routes API** (GET, POST, PUT, DELETE) manuellement,
- Vérifier les **réponses JSON** et les statuts HTTP,
- Simuler des scénarios utilisateurs (ex. : connexion, dépôt de CV, recherche de profils),
- Gérer des **collections de requêtes** classées par module,
- Exporter et partager ces collections dans l'équipe.



Grâce à Postman, nous avons pu :

- **Détecter rapidement les erreurs** ou incohérences côté serveur,
- **Valider les intégrations frontend-backend** de manière progressive,
- Accélérer le **debugging**, en testant chaque point de terminaison sans devoir passer par l'interface utilisateur.

Cette combinaison d'outils a grandement contribué à :

- Une **meilleure organisation du travail**,
- Une **communication claire** et une répartition fluide des tâches,
- Un **respect des délais** et une **productivité constante** tout au long du projet.

Chapitre 2 : État de l'art

1. Introduction

L'objectif de ce chapitre est de présenter un panorama des approches, outils et technologies existants dans le domaine du traitement automatique des CV. Il met en évidence les limitations des solutions actuelles, les apports des technologies récentes comme les modèles de langage (LLMs), ainsi que la place des APIs comme Gemini dans une architecture moderne de traitement de données non structurées.

2. Traitement automatique des CV : Défis et enjeux

Les CV sont des documents riches en informations, mais aussi très **hétérogènes** dans leur structure :

- Certains sont sous forme de texte simple, d'autres sous forme graphique ou même en colonnes.
- Les intitulés varient énormément : "Expérience", "Parcours professionnel", "Professional Background", etc.
- La langue, le style, et la densité du contenu diffèrent d'un candidat à l'autre.

Les principaux défis sont donc :

- **L'extraction fiable** d'informations à partir de documents non structurés (PDF, Word...).
- **La normalisation** des données extraites (ex : reconnaître que "Maîtrise en informatique" = "Master").
- **Le matching intelligent** entre un profil candidat et les exigences d'un poste, en tenant compte du sens et du contexte.

3. Les systèmes ATS (Applicant Tracking Systems) traditionnels

Les ATS sont des outils largement utilisés en entreprise pour gérer les candidatures. Parmi les plus connus, on trouve :

Taleo, Workday, SmartRecruiters, ou SAP SuccessFactors.

Leur fonctionnement repose souvent sur :

➤ Stockage Structuré des Données du Candidat

Au cœur des ATS traditionnels se trouve une base de données rigide. Chaque information extraite du CV (nom, expérience, formation, compétences, etc.) est rangée dans des champs spécifiques. Cela permettait une organisation claire et une recherche ciblée.

- **Avantage initial** : Centralisation de l'information, facilitant la gestion d'un grand nombre de candidatures. Fini les piles de CV papier !

- **Limitation :** Cette structure rigide signifie que si une information n'est pas placée dans le champ attendu, elle risque de ne pas être prise en compte lors des recherches. Par exemple, une compétence clé mentionnée uniquement dans une description de poste et non dans la section "compétences" pourrait être ignorée. De plus, la mise en page et la structure variées des CV peuvent rendre l'extraction automatique des données parfois hasardeuse.

➤ **Recherche par Mots-Clés Exacts**

La fonctionnalité de recherche par mots-clés était (et est toujours) un pilier des ATS. Les recruteurs définissent des mots-clés spécifiques liés au poste (par exemple, "gestion de projet", "Java", "comptabilité") et le système identifie les CV qui contiennent ces termes.

- **Avantage perçu :** Gain de temps considérable dans le tri initial des candidatures. Permet d'éliminer rapidement les profils qui ne correspondent manifestement pas aux exigences de base.
- **Limites cruciales :**
 - **Manque de compréhension sémantique :** C'est le point clé. Un ATS traditionnel ne comprend pas les synonymes, les paraphrases ou le contexte. Si un candidat utilise "chef de projet" au lieu de "gestionnaire de projet", son CV pourrait ne pas être retenu, même si les compétences sont identiques.
 - **Exigence de mots-clés précis :** Les candidats doivent deviner les termes exacts utilisés par le recruteur. Cela favorise ceux qui connaissent les "mots à la mode" ou qui adaptent artificiellement leur CV pour correspondre aux offres.
 - **Risque de faux négatifs :** Des candidats parfaitement qualifiés peuvent être écartés si leur vocabulaire ne correspond pas exactement à celui recherché.

Limites :

- **Ne comprennent pas le sens du contenu :** Comme mentionné, c'est une limitation fondamentale. L'ATS traite le texte comme une simple suite de caractères, sans saisir les nuances, les implications ou les relations entre les mots. Un CV peut être truffé de mots-clés sans pour autant refléter une réelle compétence.
- **Faible performance sur les CV non conventionnels ou créatifs :** Les CV qui sortent des formats standards (design original, utilisation de portfolios interactifs, etc.) sont souvent mal interprétés par les parsers des ATS traditionnels. Les informations peuvent être mal extraites, voire ignorées, désavantageant les profils créatifs et potentiellement très pertinents.
- **Matching souvent superficiel et peu intelligent :** Le "matching" (la mise en relation des CV avec les offres d'emploi) se limite souvent à la présence ou à l'absence de mots-clés. Il ne tient pas compte de la qualité de l'expérience, de la progression de carrière, des compétences transférables ou de l'adéquation culturelle potentielle. C'est une approche quantitative plutôt que qualitative.

4. Approches basées sur le Machine Learning

Certaines plateformes utilisent du Machine Learning (ML) et du Natural Language Processing (NLP).

➤ Approches Basées sur le Machine Learning

L'idée ici est de doter les systèmes d'une certaine forme d'intelligence pour mieux comprendre et interpréter le contenu des CV, allant au-delà de la simple présence de mots-clés.

- **Identifier les entités nommées (NER) :** Les modèles de NER sont entraînés pour reconnaître et catégoriser des éléments spécifiques dans le texte, comme les noms de personnes, les titres de postes, les noms d'entreprises, les lieux, les dates, etc.
 - **Avantage :** Permet d'extraire des informations clés de manière plus structurée et moins dépendante de la formulation exacte. Par exemple, un système peut identifier "Google" et "Alphabet Inc." comme la même entreprise.
 - **Exemple :** Un modèle de NER pourrait identifier "Ingénieur logiciel senior chez Google de 2018 à 2022" et extraire "Ingénieur logiciel senior" comme poste, "Google" comme entreprise et "2018-2022" comme période.
- **Classer les sections du CV :** Les modèles de classification sont utilisés pour identifier et catégoriser les différentes sections d'un CV, même si elles ne sont pas toujours clairement intitulées "Expérience professionnelle" ou "Formation".
 - **Avantage :** Facilite l'extraction d'informations pertinentes même lorsque la structure du CV est inhabituelle.
 - **Exemple :** Un modèle pourrait identifier une section intitulée "Ce que j'ai fait" comme correspondant à l'expérience professionnelle.
- **Recommander des offres selon le profil :** En analysant le contenu du CV et les descriptions de poste, ces systèmes peuvent suggérer des offres d'emploi potentiellement intéressantes pour le candidat.
 - **Avantage :** Améliore l'expérience candidat en lui présentant des opportunités pertinentes et peut aider les recruteurs à identifier des profils intéressants pour des postes ouverts.
 - **Fonctionnement :** Le modèle compare les compétences, l'expérience et les préférences du candidat (si disponibles) avec les exigences et les descriptions des postes.

➤ **Techniques Courantes Utilisées**

- **Représentation du texte :**

- **TF-IDF (Term Frequency-Inverse Document Frequency) :** Une méthode statistique pour évaluer l'importance d'un mot dans un document au sein d'une collection de documents (ici, les CV). Les mots fréquents dans un CV mais rares dans l'ensemble de la base de données sont considérés comme plus importants.
- **Bag-of-Words :** Un modèle simple qui représente un texte comme un ensemble de ses mots, sans tenir compte de l'ordre ou de la structure grammaticale.
- **Word2Vec :** Un modèle plus avancé qui apprend des représentations vectorielles (embeddings) pour les mots, capturant ainsi leurs significations sémantiques et leurs relations (synonymes, analogies). Par exemple, "roi" et "homme" seraient plus proches dans l'espace vectoriel que "roi" et "voiture".

- **Algorithmes de classification et de régression :**

- **SVM (Support Vector Machines) :** Un algorithme puissant pour la classification, trouvant l'hyperplan optimal qui sépare les différentes classes (par exemple, CV correspondant ou non à un poste).
- **Random Forests :** Un algorithme d'ensemble qui construit de nombreux arbres de décision et prend la décision majoritaire, souvent performant et robuste.
- **KNN (K-Nearest Neighbors) :** Un algorithme simple qui classe un nouveau point de données en se basant sur la classe majoritaire de ses k voisins les plus proches dans l'espace des caractéristiques.

- **Segmentation de séquences :**

- **Modèles CRF (Conditional Random Fields) :** Utilisés pour étiqueter des séquences de données, comme identifier les différentes parties d'une phrase ou les différentes sections d'un CV. Ils tiennent compte du contexte des mots voisins pour faire des prédictions.

- **Librairies logicielles :**

- **spaCy :** Une librairie de pointe pour le NLP industriel, offrant des modèles pré-entraînés pour diverses tâches comme la NER, la POS tagging (identification des parties du discours) et l'analyse de la dépendance.

- **Scikit-learn** : Une librairie incontournable pour le Machine Learning en Python, fournissant des outils pour la classification, la régression, le clustering, la réduction de dimensionnalité, etc.
- **Gensim** : Une librairie axée sur la modélisation de sujets (topic modeling) et la similarité de documents, incluant des implémentations de Word2Vec et d'autres techniques d'embedding.

Limites :

- **Nécessitent beaucoup de données annotées pour l'entraînement** : Pour que les modèles soient performants, ils doivent être entraînés sur de vastes quantités de données (ici, des CV) qui ont été manuellement étiquetées et annotées (par exemple, identifier les noms, les compétences, les sections). La création de ces jeux de données est coûteuse en temps et en ressources.
- **Faible capacité à comprendre le contexte** : Bien que Word2Vec et les embeddings capturent certaines relations sémantiques, la compréhension du contexte global d'une phrase ou d'un paragraphe reste un défi. L'ironie, le sarcasme, les expressions idiomatiques et les connaissances du monde réel sont difficiles à intégrer dans ces modèles.
- **Manquent de généralisation pour des CV très variés** : Les modèles entraînés sur un certain type de CV (par exemple, des profils techniques très structurés) peuvent avoir du mal à bien interpréter des CV avec des formats très différents, des parcours atypiques ou un vocabulaire non standard. La diversité des styles et des formats de CV reste un obstacle.

5. Modèles de type NER (Named Entity Recognition)

Le NER est une tâche fondamentale du NLP qui vise à identifier et à catégoriser des entités nommées dans un texte. Appliqué aux CV, cela signifie repérer et classer des informations spécifiques et pertinentes pour le recrutement.

- **Exemples d'entités : Le Vocabulaire du Recrutement Automatisé**
 - **Nom** : Identifier le nom du candidat.
 - **Email** : Extraire l'adresse électronique.
 - **Diplôme** : Reconnaître les titres de diplômes obtenus (ex : Licence en Informatique, Master en Marketing).
 - **Entreprise** : Identifier les noms des employeurs précédents.
 - **Poste** : Extraire les titres des postes occupés (ex : Chef de projet, Développeur web).
 - **Durée** : Reconnaître les périodes d'emploi ou de formation (ex : 2018-2022, Septembre 2015 - Juin 2018).
 - Et bien d'autres encore, comme les compétences techniques, les langues parlées, les certifications, etc.

Approches :

- **Modèles supervisés entraînés sur des jeux de données :**
 - L'approche la plus courante consiste à entraîner des modèles de Machine Learning sur de grands ensembles de CV où les entités ont été manuellement annotées et labellisées.
 - **Exemple de jeu de données : LayoutLM Resume Dataset.** Ce type de jeu de données est spécifiquement conçu pour l'extraction d'informations de CV et prend en compte la mise en page des documents.
 - Le modèle apprend ainsi à reconnaître les motifs et les contextes associés à chaque type d'entité.
- **Utilisation de modèles pré-entraînés :**
 - Des modèles de langage puissants comme **BERT (Bidirectional Encoder Representations from Transformers)**, **LayoutLM**, **Flair**, ou **Stanza** peuvent être utilisés pour la tâche de NER.
 - **BERT :** Un modèle transformer qui a révolutionné le NLP grâce à sa capacité à comprendre le contexte bidirectionnel des mots. Il peut être fine-tuné pour des tâches spécifiques comme le NER sur des CV.
 - **LayoutLM :** Une extension de BERT qui prend en compte non seulement le contenu textuel mais aussi la mise en page visuelle des documents. C'est particulièrement pertinent pour les CV au format PDF où la structure visuelle donne des indices importants sur les différentes sections et les relations entre les informations.
 - **Flair :** Une librairie NLP qui offre d'excellents modèles pour le NER, souvent basés sur des embeddings de mots contextuels.
 - **Stanza (anciennement StanfordNLP) :** Une autre librairie NLP robuste qui fournit des outils pour diverses tâches, y compris le NER, avec des modèles pré-entraînés pour de nombreuses langues.

Avantages :

- **Bonne précision pour des entités simples :** Pour les entités clairement définies et courantes comme les noms, les emails ou les dates, les modèles NER bien entraînés peuvent atteindre une précision élevée.
- **LayoutLM utilise la mise en page du document : Un Atout pour les PDF**

L'intégration de la mise en page par des modèles comme LayoutLM est un avantage considérable pour l'analyse des CV au format PDF, car la structure visuelle (titres de sections, listes à puces, etc.) aide grandement à identifier et à relier les informations.

Inconvénients :

- **Doivent être entraînés ou fine-tunés localement :** Coût et Expertise Pour obtenir des performances optimales sur des types de CV spécifiques à une industrie ou dans une langue particulière, il est souvent nécessaire d'entraîner ou de fine-tuner ces modèles sur des données locales. Cela requiert des compétences en Machine Learning et des ressources de calcul (GPU notamment).
- **Difficulté à traiter des documents complexes ou dans des langues multiples :** La Variété comme Obstacle Les CV avec des mises en page très complexes, des formats non standard ou rédigés dans des langues pour lesquelles il n'existe pas de modèles pré-entraînés performants peuvent poser des défis significatifs pour les modèles NER. La généralisation à des langues multiples nécessite des modèles multilingues ou des entraînements spécifiques pour chaque langue.

6. Apport des LLMs (Large Language Models)

Des modèles comme GPT (OpenAI), Gemini (Google), Claude (Anthropic) ou LLaMA (Meta) ont démontré une capacité impressionnante à comprendre et à générer du langage naturel. Leur application à l'analyse des CV ouvre des perspectives fascinantes.

Fonction	Capacité d'un LLM
Compréhension sémantique	Peut lire un CV et en déduire les points forts
Résumé automatique	Génère un résumé professionnel à partir d'un profil
Extraction structurée	Donne un JSON contenant les sections clés
Matching intelligent	Évalue la compatibilité entre un CV et une offre

Grâce à des prompts bien formulés, ces modèles peuvent :

- **Extraire toutes les données importantes d'un CV en une seule requête :** Au lieu de s'appuyer sur des modèles NER entraînés spécifiquement pour chaque type d'entité, un LLM peut, avec une instruction claire, identifier et extraire une grande variété d'informations pertinentes d'un CV.
- **Fournir une évaluation qualitative d'une candidature :** Les LLMs peuvent aller au-delà de la simple présence de mots-clés et évaluer la qualité de l'expérience, la progression de carrière, les réalisations et l'adéquation globale du profil au poste.
- **Justifier leur décision (matching, points forts / points faibles) :** Contrairement aux systèmes traditionnels qui donnent un score de matching sans explication, un LLM peut argumenter sa décision en mettant en évidence les compétences et l'expérience pertinentes ou les points faibles potentiels.

a- Gemini (Google AI)

Gemini, développé par Google DeepMind, représente une avancée significative dans les modèles de langage. Ses capacités le rendent particulièrement pertinent pour l'analyse de CV

- **Lire et comprendre de longs textes :** Les CV, qui peuvent être longs et détaillés, sont parfaitement adaptés à la capacité de Gemini à traiter des séquences de texte étendues. Il peut saisir les informations clés même au sein de descriptions de poste complexes.
- **Répondre à des instructions complexes avec des sorties formatées :** La possibilité de demander à Gemini de fournir des résultats structurés en JSON, tableaux ou listes facilite l'intégration de l'analyse dans des systèmes automatisés.
- **Comprendre des subtilités linguistiques, des compétences implicites et des styles variés :** C'est un avantage majeur par rapport aux approches basées sur des mots-clés ou des modèles NER classiques. Gemini peut interpréter le sens derrière les formulations, identifier des compétences qui ne sont pas explicitement mentionnées et s'adapter à différents styles de rédaction.

7. Comparaison des approches

Critère	ATS classiques	Modèles ML locaux	LLMs via API (ex: Gemini)
Précision sur CV variés	Faible	Moyenne	Élevée
Complexité d'implémentation	Faible	Élevée	Moyenne
Coût (infrastructure)	Moyen	Élevé (GPU, data)	Moyen (via quota API)
Personnalisation	Faible	Élevée	Moyenne à Élevée
Temps de mise en œuvre	Rapide	Long	Rapide
Extraction sémantique	Non	Partiellement	Oui (très avancée)
Matching intelligent	Basé sur mots-clés	Possible	Oui (avec justification)

En résumé, l'arrivée des LLMs comme Gemini ouvre un champ de possibilités immense pour l'analyse des CV. Leur capacité à comprendre le langage naturel de manière sophistiquée permet de dépasser les limitations des approches traditionnelles et d'obtenir des informations plus riches et plus pertinentes pour le processus de recrutement. L'absence de besoin d'entraînement spécifique et la facilité d'intégration via API en font des outils particulièrement attractifs pour les entreprises.

Pourquoi Gemini a été choisi dans votre projet ? Les Atouts Majeurs

- **Il ne nécessite aucun entraînement ou dataset annoté :** C'est une rupture par rapport aux modèles supervisés comme ceux utilisés en NER, qui demandent un investissement considérable en temps et en ressources pour la création et l'annotation de données. Gemini comprend les instructions directement via le prompting.
- **Il est accessible via une API à intégrer facilement dans une application web :** L'intégration via une API simplifie le développement et le déploiement de solutions d'analyse de CV basées sur Gemini.
- **Il fournit des résultats immédiats et adaptables en fonction de la formulation des prompts :** La flexibilité des prompts permet d'adapter l'analyse aux besoins spécifiques de chaque poste ou de chaque entreprise. On peut affiner les instructions pour obtenir des informations très ciblées.

8. Conclusion

L'analyse de l'état de l'art met en évidence que les solutions traditionnelles ne suffisent plus à répondre aux besoins actuels en recrutement intelligent. Les modèles locaux nécessitent des efforts considérables en annotation et en infrastructure. Les LLMs, comme **Gemini**, représentent une **révolution** : ils permettent une **automatisation intelligente, rapide et contextuelle** du traitement des CV, sans passer par l'entraînement classique.

Le choix d'intégrer Gemini dans une architecture MERN avec un module Python intermédiaire permet de **bénéficier de l'intelligence des modèles IA tout en conservant la maîtrise sur les données et le flux d'utilisation.**

Chapitre 3 : Analyse des besoins (Cahier des charges)

1. Introduction

Dans ce chapitre, nous définissons les besoins fonctionnels et non fonctionnels du système, ainsi que les contraintes techniques et organisationnelles. Cette analyse vise à garantir que la solution réponde aux attentes des utilisateurs finaux (candidats et recruteurs) tout en respectant les limites du cadre technique (technologies utilisées, API Gemini, sécurité, etc.).

2. Présentation du système à concevoir

Une plateforme web intelligente dédiée à la gestion des CV. Les fonctionnalités principales pour chaque type d'utilisateur sont introduites. L'originalité du système est soulignée, mettant en avant l'utilisation de l'IA (via Gemini) pour l'extraction et le matching sémantique, ainsi que la création dynamique d'une base de données structurée.

Points Clés :

- **Fonctionnalités clés pour les candidats :** Création de compte, dépôt de CV (PDF), visualisation du profil analysé.
- **Fonctionnalités clés pour les recruteurs :** Création de compte, publication d'offres, consultation des profils pertinents proposés par l'IA.
- **Éléments d'originalité :**
 - Extraction intelligente via API Gemini et "Prompt Engineering".
 - Matching sémantique (compréhension du sens).
 - Base de données dynamique alimentée par l'IA.

3. Acteurs du système

Acteur	Rôle	Accès
Candidat	Déposer un CV, voir ses données extraites, consulter les offres d'emploi	Interface utilisateur
Recruteur	Créer une offre, visualiser les profils recommandés par l'IA	Interface RH
Système IA (Gemini)	Extraire, structurer et analyser le contenu d'un CV ou d'une offre	Via API

4. Besoins fonctionnels (BF)

Les besoins fonctionnels décrivent les actions spécifiques que le système doit être capable de réaliser. Ils sont numérotés pour faciliter leur suivi.

- **BF1. Authentification & Gestion des comptes :**

- **Description :** Gestion de l'inscription et de la connexion pour les candidats et les recruteurs, sécurité des mots de passe et gestion des sessions via JWT (JSON Web Tokens).
- **Importance :** Fonctionnalité fondamentale pour sécuriser l'accès au système et gérer les identités des utilisateurs.

- **BF2. Dépôt de CV (Candidat) :**

- **Description :** Permettre l'upload de CV au format PDF, extraction du texte brut à l'aide de bibliothèques Python spécifiques, envoi du texte à l'API Gemini via un prompt (instruction), récupération des données structurées au format JSON (Nom, Email, Compétences, etc.), et sauvegarde de ces données dans la base de données MongoDB.
- **Importance :** Fonctionnalité clé pour l'intégration des données des candidats dans le système. L'utilisation de Gemini pour l'extraction est un point central de l'originalité du système.

- **BF3. Création d'une offre d'emploi (Recruteur) :**

- **Description :** Fournir un formulaire pour permettre aux recruteurs de créer des offres d'emploi en spécifiant les informations pertinentes (poste, niveau, compétences requises, expérience, etc.) et sauvegarde de ces offres dans MongoDB.
- **Importance :** Fonctionnalité essentielle pour permettre aux recruteurs de publier les postes à pourvoir.

- **BF4. Matching des candidats avec une offre :**

- **Description :** Envoyer le contenu de l'offre d'emploi et les données extraites des CV à Gemini via un prompt, générer une liste de candidats pertinents classés par un score de compatibilité, et afficher cette liste côté recruteur.
- **Importance :** Fonctionnalité centrale de la valeur ajoutée du système, utilisant l'IA pour faciliter la tâche des recruteurs en leur présentant les profils les plus adaptés.

- **BF5. Interfaces utilisateurs :**

- **Description :** Définir les éléments clés des interfaces pour les candidats (profil, CV déposé, données extraites) et les recruteurs (liste des offres, candidats proposés par l'IA). L'accent est mis sur une conception responsive et intuitive.
- **Importance :** L'ergonomie et l'expérience utilisateur sont cruciales pour l'adoption et l'efficacité de la plateforme.

5. Besoins non fonctionnels (BNF)

Les besoins non fonctionnels décrivent les qualités que le système doit posséder, en se concentrant sur la sécurité, la performance, l'évolutivité et l'accessibilité.

- **BNF1. Sécurité :**

- **Description :** Assurer le chiffrement des données sensibles (notamment les mots de passe) et sécuriser les communications via l'utilisation de JWT.
- **Importance :** Indispensable pour protéger les données des utilisateurs et garantir la confidentialité.

- **BNF2. Performance :**

- **Description :** Assurer une extraction rapide des informations par l'IA et un affichage fluide des données, même en présence d'un grand nombre d'offres et de candidats.
- **Importance :** Impacte directement l'expérience utilisateur et l'efficacité du système.

- **BNF3. Évolutivité :**

- **Description :** Concevoir le système de manière à faciliter l'ajout de nouvelles fonctionnalités futures (chat IA, analyse linguistique approfondie, historique des interactions, etc.).
- **Importance :** Permet au système de s'adapter aux besoins futurs et d'intégrer des innovations.

- **BNF4. Accessibilité :**

- **Description :** Assurer la conformité avec les bonnes pratiques en matière d'expérience utilisateur (UX) et d'interface utilisateur (UI), rendant le système facile à utiliser pour tous.
- **Importance :** Contribue à une adoption plus large et à une meilleure satisfaction des utilisateurs.

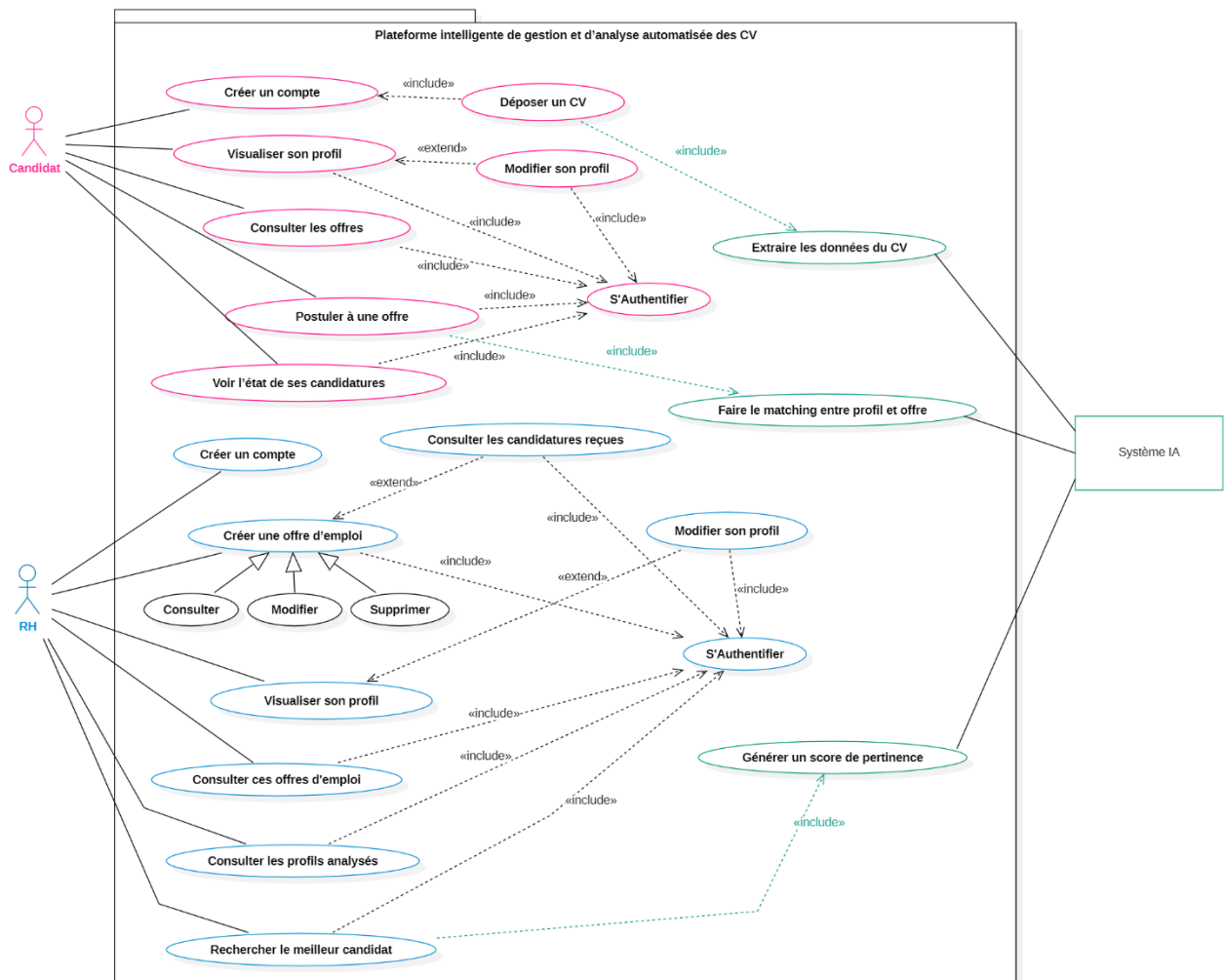
6. Contraintes techniques

- **Langages :** JavaScript (pour le frontend et le backend) et Python (principalement pour le traitement PDF et l'interaction avec l'API Gemini).
- **Frontend :** La librairie ReactJS pour la construction de l'interface utilisateur et Tailwind CSS pour le stylage.
- **Backend :** Le framework Node.js avec Express pour la création de l'API REST.
- **Base de données :** MongoDB, une base de données NoSQL orientée documents.
- **Extraction IA :** L'API de Google Gemini sera utilisée comme moteur d'intelligence artificielle.
- **Traitement PDF :** Des librairies Python comme PDFMiner ou PyMuPDF seront utilisées pour extraire le texte des CV.

- **Hébergement** : Pour l'instant, l'hébergement est prévu en local (pour le développement).
- **Architecture** : Le système suivra une architecture de type MERN (MongoDB, Express, React, Node.js) avec une couche Python dédiée à l'interaction avec l'API Gemini.

7. Diagramme de cas d'utilisation (Use Case)

Ce diagramme de cas d'utilisation fournit une vue d'ensemble claire des fonctionnalités offertes par la Plateforme et des interactions entre les différents acteurs.



• Analyse du Diagramme de Cas d'Utilisation

Le diagramme de cas d'utilisation présenté ci-dessus décrit les fonctionnalités principales de la "Plateforme Intelligente de gestion et d'analyse automatisée des CV". Il met en évidence les interactions entre les acteurs du système (le Candidat et le Responsable des Ressources Humaines - RH) et les cas d'utilisation qu'ils initient, ainsi que le rôle du Système d'Intelligence Artificielle (IA) en tant qu'acteur secondaire participant à certains processus clés.

➤ Acteurs du Système

Deux acteurs principaux interagissent avec la plateforme :

- **Candidat** : Représente tout utilisateur souhaitant créer un profil, déposer son curriculum vitae, consulter les offres d'emploi et postuler à celles qui correspondent à son profil.
- **Responsable des Ressources Humaines (RH)** : Représente tout utilisateur autorisé à créer et gérer des offres d'emploi, consulter les candidatures reçues, analyser les profils des candidats et rechercher les meilleurs talents.
- **Système IA** : Représente le composant intelligent de la plateforme, responsable de l'extraction des données des CV, du matching entre les profils et les offres d'emploi, et de la génération de scores de pertinence.

➤ Cas d'Utilisation Détaillés

Les cas d'utilisation suivants décrivent les fonctionnalités offertes par la plateforme :

Cas d'Utilisation Initiés par le Candidat

1. **Créer un compte** : Ce cas d'utilisation permet à un nouvel utilisateur de s'enregistrer sur la plateforme en fournissant les informations d'identification nécessaires. Il constitue la première étape pour interagir avec le système.
2. **Déposer un CV (<<include>> S'Authentifier)** : Une fois authentifié, le candidat peut télécharger son curriculum vitae au format numérique. Ce document est ensuite traité par le Système IA pour l'extraction des données.
3. **Visualiser son profil (<<include>> S'Authentifier, <<extend>> Modifier son profil)** : Le candidat connecté a la possibilité de consulter les informations de son compte, incluant potentiellement les données extraites de son CV. Il peut également étendre ce cas d'utilisation en choisissant de modifier son profil.
4. **Modifier son profil (<<include>> S'Authentifier, <<include>> Extraire les données du CV)** : Ce cas d'utilisation permet au candidat de mettre à jour les informations de son compte et de potentiellement soumettre une nouvelle version de son CV, déclenchant une nouvelle extraction de données par le Système IA.
5. **Consulter les offres (<<include>> S'Authentifier - conditionnel)** : Le candidat peut parcourir les offres d'emploi disponibles sur la plateforme. L'authentification peut être requise en fonction des paramètres d'accès aux offres.
6. **Postuler à une offre (<<include>> S'Authentifier, <<include>> Consulter les offres)** : Après avoir consulté une offre, le candidat connecté peut soumettre sa candidature, associant ainsi son profil et son CV à l'offre sélectionnée.
7. **Voir l'état de ses candidatures (<<include>> S'Authentifier)** : Le candidat connecté peut suivre l'avancement de ses candidatures pour les différents postes auxquels il a postulé.

Cas d'Utilisation Initiés par le RH

1. **Créer un compte** : Ce cas d'utilisation permet à un nouvel utilisateur avec le rôle de RH de s'inscrire sur la plateforme, avec les autorisations spécifiques à son rôle.
2. **Créer une offre d'emploi (<<include>> S'Authentifier)** : Le RH connecté peut rédiger et publier de nouvelles offres d'emploi, en spécifiant les détails du poste et les compétences recherchées.
3. **Consulter ses offres d'emploi (<<include>> S'Authentifier, <<extend>> Modifier, <<extend>> Supprimer)** : Le RH connecté peut visualiser et gérer les offres d'emploi qu'il a créées. Il a également la possibilité de modifier ou de supprimer des offres existantes.
4. **Consulter les candidatures reçues (<<include>> S'Authentifier, <<include>> Extraire les données du CV, implication du Système IA pour Faire le matching profil et offre et Générer un score de pertinence)** : Le RH connecté peut accéder à la liste des candidats ayant postulé à ses offres. La visualisation des candidatures peut impliquer la consultation des données extraites des CV par le Système IA, ainsi que les résultats du matching et les scores de pertinence.
5. **Consulter les profils analysés (<<include>> S'Authentifier, implication du Système IA pour Extraire les données du CV)** : Le RH connecté peut consulter les informations structurées extraites des CV des candidats par le Système IA, facilitant ainsi l'évaluation rapide des compétences et de l'expérience.
6. **Rechercher le meilleur candidat (<<include>> S'Authentifier, implication du Système IA pour Faire le matching profil et offre et Générer un score de pertinence)** : Le RH connecté peut utiliser des critères de recherche pour identifier les candidats les plus pertinents. Le Système IA intervient pour effectuer le matching et générer des scores de pertinence afin de faciliter cette recherche.

Cas d'Utilisation Impliquant le Système IA

1. **Extraire les données du CV** : Ce cas d'utilisation, inclus dans plusieurs interactions initiées par le Candidat et le RH, représente le processus par lequel le Système IA analyse le contenu des CV téléchargés pour en extraire les informations structurées.
2. **Faire le matching profil et offre** : Initié en arrière-plan lors de la consultation des candidatures ou de la recherche de candidats, ce cas d'utilisation décrit la comparaison automatisée effectuée par le Système IA entre les informations du profil d'un candidat et les exigences d'une offre d'emploi.
3. **Générer un score de pertinence** : Également exécuté en arrière-plan, ce cas d'utilisation représente l'attribution d'un score par le Système IA pour quantifier l'adéquation d'un candidat à une offre d'emploi ou à des critères de recherche.

➤ Relations entre les Cas d'Utilisation

Le diagramme utilise les stéréotypes `<<include>>` et `<<extend>>` pour illustrer les relations entre les cas d'utilisation :

- `<<include>>` indique qu'un cas d'utilisation (la source de la flèche) inclut inconditionnellement l'exécution d'un autre cas d'utilisation (la cible de la flèche).
- `<<extend>>` indique qu'un cas d'utilisation (la source de la flèche) peut étendre le comportement d'un autre cas d'utilisation (la cible de la flèche) sous certaines conditions.

8. Conclusion

Cette analyse fonctionnelle et technique pose les bases du développement de la plateforme intelligente. Elle permet de cadrer les objectifs du projet, les besoins des utilisateurs, les contraintes imposées par les technologies choisies (API Gemini, MERN stack), et ouvre la voie à la phase de **modélisation UML** et **conception technique**.